

LaraClassifier - Fork + Modules Strategija

Kompletno uputstvo kako da koristiš LaraClassifier sa mogućnošću redovnog ažuriranja i custom funkcionalnosti.

Faza 1: Fork i Inicijalno Podešavanje

Korak 1: Napravi Fork na GitHub-u

1. Idi na <https://github.com/LaraClassifier/LaraClassifier>
2. Klikni dugme **"Fork"** (gore desno)
3. GitHub će kreirati kopiju na tvom nalogu: `https://github.com/tvoj-nalog/LaraClassifier`

Korak 2: Clone svoj Fork lokalno

```
bash

# Clone-uj SVOJ fork (ne original)
git clone https://github.com/tvoj-nalog/LaraClassifier.git moj-projekat

cd moj-projekat
```

Korak 3: Dodaj Original kao "upstream"

Ovo ti omogućava da povlačiš izmene iz originalnog projekta:

```
bash

# Dodaj original LaraClassifier kao "upstream" remote
git remote add upstream https://github.com/LaraClassifier/LaraClassifier.git

# Proveri da li je uspešno dodato
git remote -v

# Trebalo bi da vidiš:
# origin https://github.com/tvoj-nalog/LaraClassifier.git (fetch)
# origin https://github.com/tvoj-nalog/LaraClassifier.git (push)
# upstream https://github.com/LaraClassifier/LaraClassifier.git (fetch)
# upstream https://github.com/LaraClassifier/LaraClassifier.git (push)
```

Objašnjenje:

- `origin` = tvoj fork (gde push-uješ svoje izmene)
- `upstream` = original LaraClassifier (odakle pull-uješ ažuriranja)

Korak 4: Kreiraj Branch Strukturu

```
bash

# Kreiraj production branch za svoj kod
git checkout -b production

# Push-uješ ga na svoj fork
git push -u origin production
```

Branch strategija:

- `master` - zadržavaš sync sa upstream (original LaraClassifier)
 - `production` - tvoj aktivni projekat sa custom modulima
-

Faza 2: Instalacija i Podešavanje

Korak 5: Instaliraj Dependencies

```
bash

composer install
npm install

# Kopiraj .env fajl
cp .env.example .env

# Generiši APP_KEY
php artisan key:generate

# Podesi bazu u .env fajlu, zatim:
php artisan migrate
```

Korak 6: Konfiguriši Modules

LaraClassifier već koristi `nwidart/laravel-modules`. Proveri `config/modules.php`:

```
php
```

```
// Trebalo bi da postoji config/modules.php
```

```
// Ako ne postoji, instaliraj:
```

```
composer require nwidart/laravel-modules
```

```
php artisan vendor:publish --provider="Nwidart\Modules\LaravelModulesServiceProvider"
```

Faza 3: Kreiranje Custom Modula

Korak 7: Napravi Svoj Prvi Modul

```
bash
```

```
# Kreiraj custom modul
```

```
php artisan module:make MojeCustomFunkcije
```

```
# Laravel će kreirati strukturu:
```

```
# Modules/
```

```
# └─ MojeCustomFunkcije/
```

```
#   └─ Config/
```

```
#   └─ Console/
```

```
#   └─ Database/
```

```
#   └─ Entities/    # Modeli
```

```
#   └─ Http/
```

```
#   | └─ Controllers/
```

```
#   | └─ Middleware/
```

```
#   | └─ Requests/
```

```
#   └─ Providers/
```

```
#   └─ Resources/
```

```
#   | └─ assets/
```

```
#   | └─ views/
```

```
#   └─ Routes/
```

```
#   | └─ api.php
```

```
#   | └─ web.php
```

```
#   └─ Tests/
```

```
#   └─ composer.json
```

```
#   └─ module.json
```

Korak 8: Primer Custom Controllera u Modulu

php

// Modules/MojeCustomFunkcije/Http/Controllers/CustomController.php

```
namespace Modules\MojeCustomFunkcije\Http\Controllers;
```

```
use Illuminate\Http\Request;
```

```
use Illuminate\Routing\Controller;
```

```
class CustomController extends Controller
```

```
{
```

```
    public function index()
```

```
    {
```

```
        return view('mojecustomfunkcije::index');
```

```
    }
```

```
    public function mojaNovaFunkcionalnost()
```

```
    {
```

```
        // Tvoja custom logika ovde
```

```
        return response()->json(['status' => 'success']);
```

```
    }
```

```
}
```

Korak 9: Dodaj Routes u Modulu

php

// Modules/MojeCustomFunkcije/Routes/web.php

```
Route::prefix('custom')->group(function() {
```

```
    Route::get('/', 'CustomController@index')->name('custom.index');
```

```
    Route::post('/nova-funkcija', 'CustomController@mojaNovaFunkcionalnost');
```

```
});
```

Korak 10: Aktiviraj Modul

bash

```
# Aktiviraj modul
```

```
php artisan module:enable MojeCustomFunkcije
```

```
# Proveri status
```

```
php artisan module:list
```

Faza 4: Git Tracking Strategija

Korak 11: Konfiguriši .gitignore

Hoćeš da track-uješ **samo svoje custom stvari**, ne ceo LaraClassifier.

```
bash

# Otvori .gitignore i dodaj ovo NA POČETAK fajla:

# Ignoriši SVE po defaultu
/*

# Ali track-uj ove foldere/fajlove:
!/Modules/
!/public/custom/
!composer.json
!composer.lock
!package.json
!.env.example
!.gitignore
!README.md

# Dodatno ignoriši vendor i node_modules
/vendor/
/node_modules/
```

Objašnjenje:

- `/*` = ignoriši sve
- `!/Modules/` = OSIM Modules foldera (tvoji custom moduli)
- `!/public/custom/` = OSIM custom assets foldera

Korak 12: Commit Svoje Izmene

```
bash
```

```
# Dodaj samo svoje module
```

```
git add Modules/
```

```
git add .gitignore
```

```
git add composer.json
```

```
# Commit
```

```
git commit -m "Dodao MojeCustomFunkcije modul"
```

```
# Push na svoj fork (production branch)
```

```
git push origin production
```

Faza 5: Ažuriranje LaraClassifier-a (Glavni Deo!)

Kada izađe nova verzija LaraClassifier-a, evo kako ažuriraš:

Korak 13: Pull Izmene sa Upstream

```
bash
```

```
# 1. Prebaci se na master branch
```

```
git checkout master
```

```
# 2. Pull izmene sa originalnog LaraClassifier-a
```

```
git fetch upstream
```

```
git merge upstream/master
```

```
# 3. Push ažurirani master na svoj fork
```

```
git push origin master
```

Korak 14: Merge u Production Branch

```
bash
```

```
# 1. Prebaci se na production
```

```
git checkout production
```

```
# 2. Merge master (koji ima nove izmene) u production
```

```
git merge master
```

```
# 3. Ako ima konfliktata (retko jer su tvoje stvari u Modules/):
```

```
# - Otvori fajl sa konfliktom
```

```
# - Razreši konflikt
```

```
# - git add <fajl>
```

```
# - git commit
```

```
# 4. Push ažuriran production
```

```
git push origin production
```

Korak 15: Update Dependencies

```
bash
```

```
composer update
```

```
npm update
```

```
# Pokreni nove migracije ako ih ima
```

```
php artisan migrate
```

```
# Clear cache
```

```
php artisan cache:clear
```

```
php artisan config:clear
```

```
php artisan view:clear
```

Bonus: Deployment Strategija

Na Serveru:

```
bash
```

```
# Clone production branch
```

```
git clone -b production https://github.com/tvoj-nalog/LaraClassifier.git
```

```
cd LaraClassifier
```

```
# Install dependencies
```

```
composer install --optimize-autoloader --no-dev
```

```
npm install --production
```

```
npm run build
```

```
# Setup
```

```
cp .env.example .env
```

```
# Edituj .env sa production podešavanjima
```

```
php artisan key:generate
```

```
php artisan migrate --force
```

```
php artisan storage:link
```

```
php artisan config:cache
```

```
php artisan route:cache
```

```
php artisan view:cache
```

Kasnije za Update na Serveru:

```
bash
```

```
# Pull najnovije izmene
```

```
git pull origin production
```

```
# Update dependencies
```

```
composer install --optimize-autoloader --no-dev
```

```
npm install --production
```

```
npm run build
```

```
# Migrate i clear
```

```
php artisan migrate --force
```

```
php artisan cache:clear
```

```
php artisan config:cache
```

```
php artisan route:cache
```

```
php artisan view:cache
```

Česta Pitanja

Q: Šta ako želim da override-ujem postojeći LaraClassifier view?

Možeš u svom modulu:

```
php
// Modules/MojeCustomFunkcije/Providers/MojeCustomFunkcijeServiceProvider.php

public function boot()
{
    // Override postojećih views
    $this->loadViewsFrom(__DIR__.'/../Resources/views', 'laraclassifier');
}
```

Zatim kreiraj view sa **istim imenom** u `Modules/MojeCustomFunkcije/Resources/views/`.

Q: Mogu li da ekstenzujem postojeći LaraClassifier model?

Da! U svom modulu:

```
php
// Modules/MojeCustomFunkcije/Entities/ExtendedPost.php

namespace Modules\MojeCustomFunkcije\Entities;

use App\Models\Post;

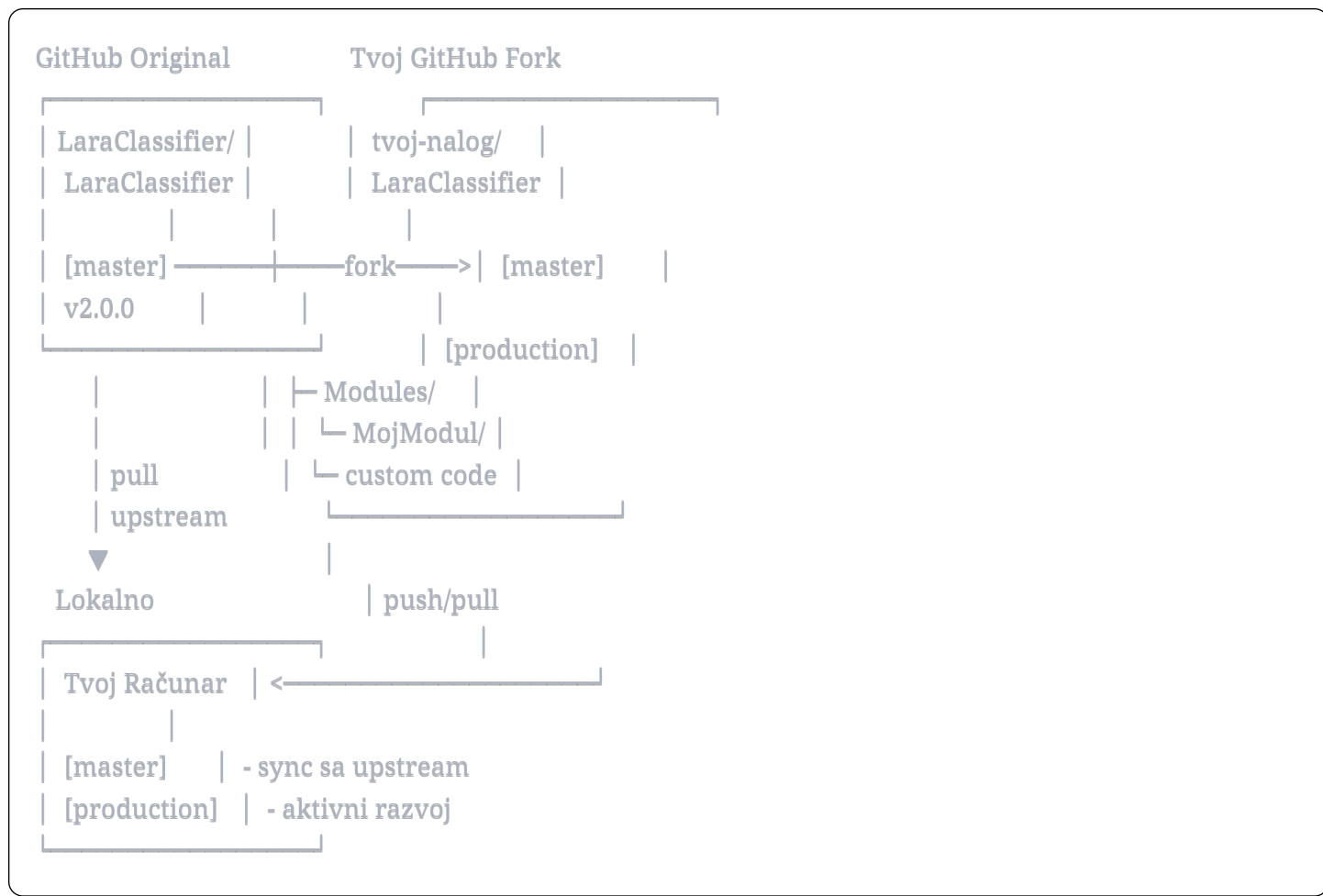
class ExtendedPost extends Post
{
    public function mojaNovaMetoda()
    {
        // Custom funkcionalnost
    }
}
```

Q: Šta ako LaraClassifier doda novi fajl koji je u konfliktu sa mojim?

Retko se dešava jer su tvoje stvari u `Modules/`. Ako se desi:

1. Tokom merge-a git će prijaviti konflikt
 2. Reši ga ručno ili preimenuj svoj fajl
 3. Nastavi sa merge-om
-

Vizualizacija Strategije



Zaključak

Ova strategija ti daje:

- ✅ **Jasnu separaciju** - sav tvoj kod je u `Modules/`
- ✅ **Lako ažuriranje** - `git pull upstream master`
- ✅ **Verzionisanje** - čuvaš istoriju svojih izmena
- ✅ **Tim rad** - drugi developeri jasno vide šta je custom
- ✅ **Deployment** - jednostavan `git pull` na serveru

Srećan coding! 🚀